CSC3060 Computer System

Project3: Part3 Code Scheduling

Ming Wen

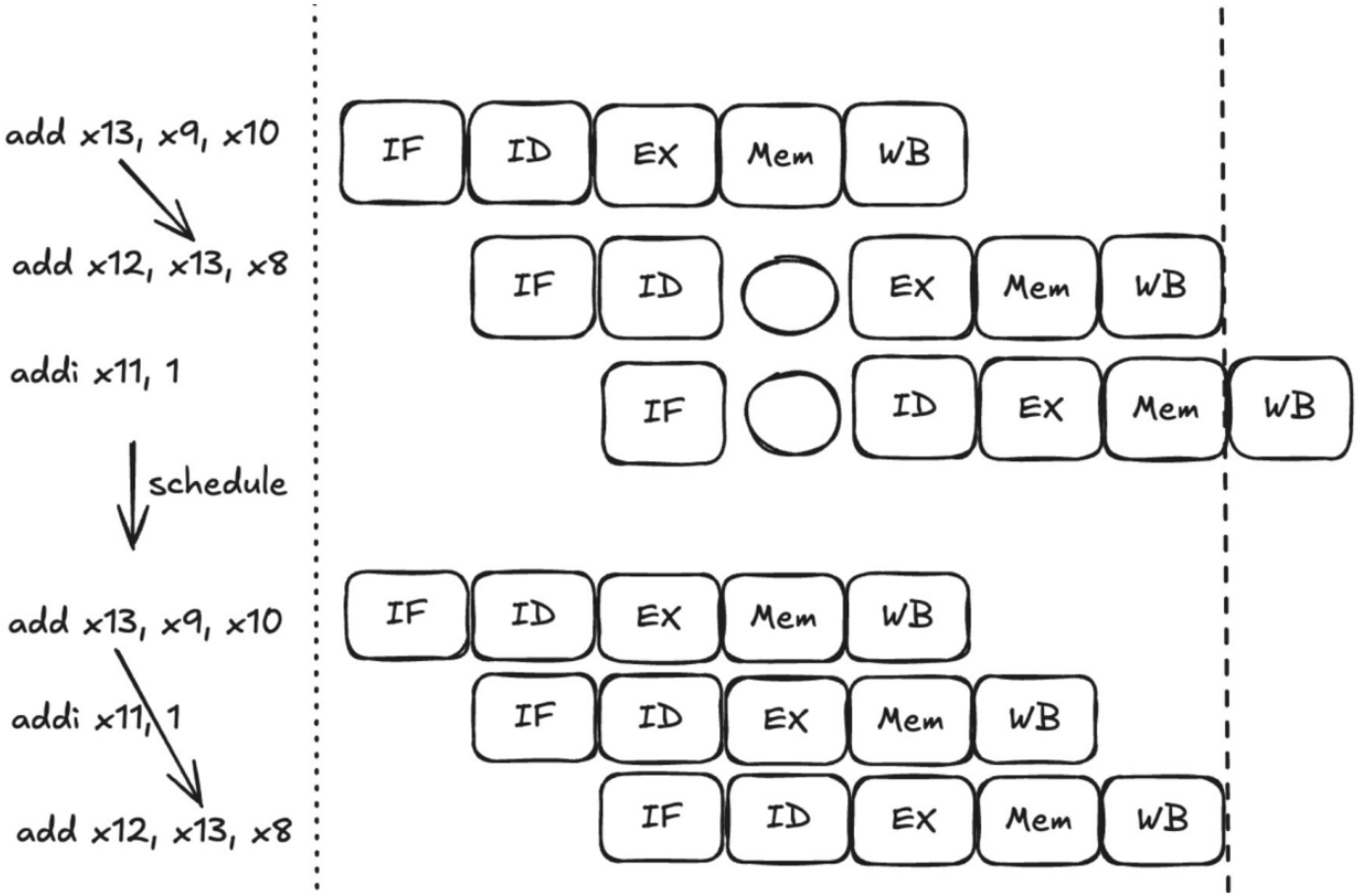
March 12th 2026

Outline

1. Code scheduling
    1. Motivation
    2. Dependency Graph
    3. Priority Calculation
    4. Scheduling process
2. Q&A

Motivation

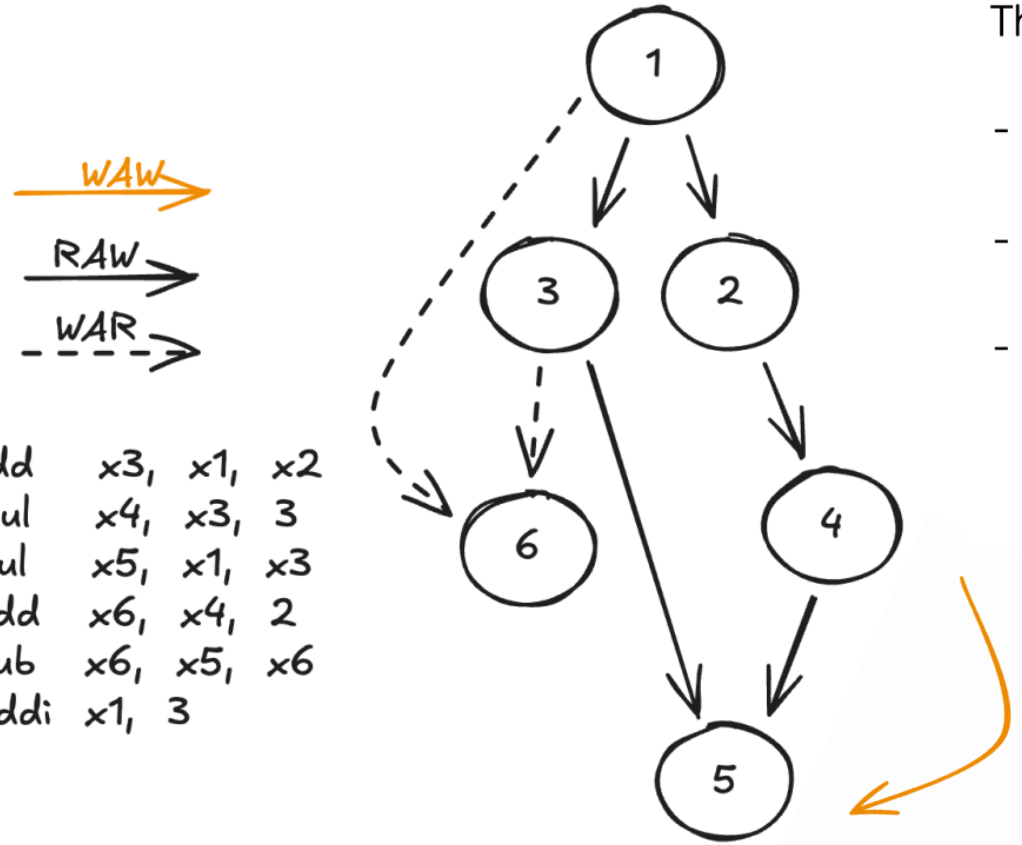
Pipeline have many bubbles because instructions are waiting for data,
so **compiler** usually will reschedule code such that this waiting can be hide by executing other instruction



Dependency Constraint

There are some dependency need to be satisfied so that after scheduling, the result is still correct
- can't consume a result before it is produced
- ambiguous dependences create many challenges

Most of the time, we can build **Dependency Graph** to present all dependency constraints



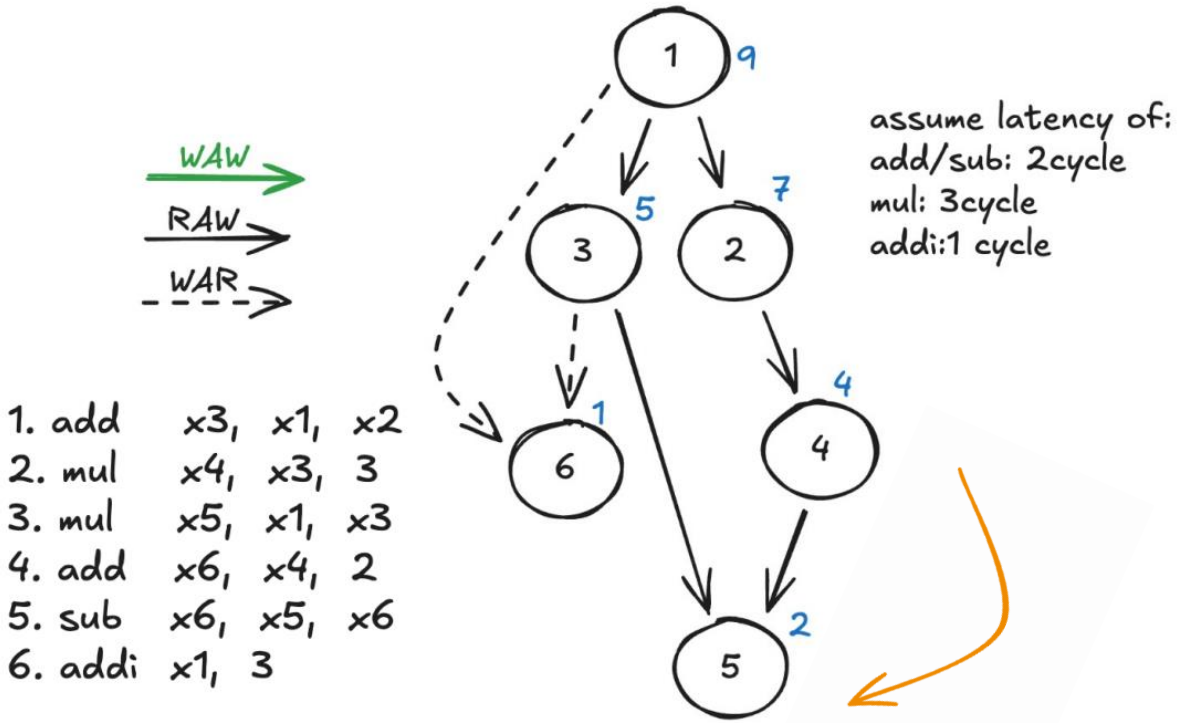
Theoretically, there are 3 type of data dependency:

- WAR (write after read) Anti-dependency
- RAW (read after write) flow-dependency
- WAW (write after write) output-dependency

Calculate Priority

$$\mathrm{prio}(x) = \begin{cases} \mathrm{latency}(x), & \text{if } x \text{ is a leaf,} \\ \max\left(\mathrm{latency}(x) + \max_{(x,y)\in E}\left(\mathrm{prio}(y)\right),\ \max_{(x,y)\in E'}\left(\mathrm{prio}(y)\right)\right), & \text{otherwise.} \end{cases}$$

Here (x, y) in E means x->y is a flow dependency (RAW)
(x, y) in E' means x->y is a anti-dependency (WAR) or output-dependency(WAW)



Scheduling

Initially we put all free instructions into a `ready-list`
at each iteration:

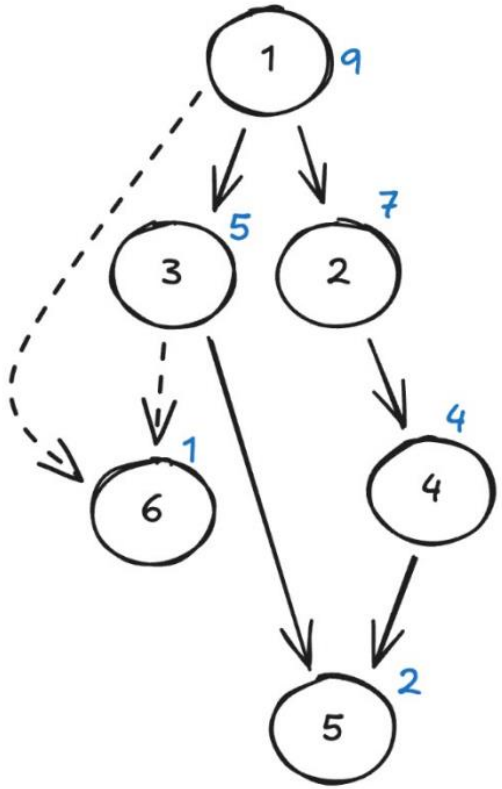
pick out a instruction from `ready-list` follow some 'rule'
add all new free instruction into `ready-list`

If we always prefer to choose instruction with larger priority:

assume latency of:
add/sub: 2cycle
mul: 3cycle
addi:1 cycle

RAW
WAR

1. add x3, x1, x2
2. mul x4, x3, 3
3. mul x5, x1, x3
4. add x6, x4, 2
5. sub x6, x5, x6
6. addi x1, 3



cycle1: {1} pick (1)
cycle2: {} empty
cycle3: {2,3} pick(2)
cycle4: {3} pick(3)
cycle5: {6} pick(6)
cycle6: {4} pick(4)
cycle7: {} empty
cycle8: {5} pick(5)

After rescheduling:
123645

What if there is a Tie Condition?

There are many other heuristic rules to follow:

- Favor the **original order**
- Favor nodes that have a **large number of children**
- Favor nodes that are the **final use of a regester**

As long as the final sequence satisfy topological order, it is a valid scheduling(may not optimal)

Thanks!